MULTI-TEXTURE BUILDING SURFACE CRACK DETECTION USING VISION-BASED DEEP LEARNING AND INTERPRETABILITY TECHNIQUES

PROJECT REPORT

Submitted by

GEORGE PRIJI PROMOD

(Reg. No. MBT24CSCE04, M24CS1003)

to

Mar Baselios College of Engineering and Technology

towards partial fulfillment of the requirements for the award of

the Degree of Master of Technology in Computer Science & Engineering

by

the APJ Abdul Kalam Technological University

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MAR BASELIOS COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Mar Ivanios Vidyanagar, Thiruvananthapuram - 695 015

APRIL 2026

DECLARATION

I hereby declare that the report of mini project "**Multi-Texture Building Surface Crack Detection Using Vision-Based Deep Learning and Interpretability Techniques**", being submitted towards partial fulfillment of the requirements for the award of degree of Master of Technology in Computer Science & Engineering of the APJ Abdul Kalam Technological University is a bonafide work done by me under the supervision of **Dr. Anne Dickson**, Associate Professor. This submission is written in my own words and has been adequately and accurately cited as appropriate. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the college and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma or similar titles of any University.

Trivandrum — George Priji Promod

April 28, 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MAR BASELIOS COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

THIRUVANANTHAPURAM – 695 015

CERTIFICATE

This is to certify that the report titled **MULTI-TEXTURE BUILDING SURFACE CRACK DETECTION USING VISION-BASED DEEP LEARNING AND INTERPRETABILITY TECHNIQUES** submitted by George Priji Promod (Reg. No. **MBT24CSCE04, M24CS1003**) to Mar Baselios College of Engineering and Technology towards partial fulfillment of the requirements for the award of the Degree of Master of Technology in Computer Science & Engineering with specialization by the APJ Abdul Kalam Technological University is a bonafide record of the mini project work carried out by him/her under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for the award of any degree, diploma or similar titles.

Dr. Anne Dickson
Associate Professor
(Guide)

Dr. Jesna Mohan
Associate Professor
(Project Coordinator)

Head of the Department

April 28, 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have supported and guided me during the initial phase of this project. Their encouragement and assistance have played a significant role in shaping the progress made so far.

I extend my heartfelt thanks to my project guide, Dr. Anne Dickson, Associate Professor, Department of Computer Science and Engineering, for her valuable guidance, insightful suggestions, and continuous support throughout the development of this work. Her expertise and motivation have been instrumental in helping me navigate this phase of the project.

I am also grateful to my project coordinator, Dr. Jesna Mohan, Associate Professor, Department of Computer Science and Engineering, for her timely feedback, helpful advice, and consistent encouragement during the course of this project.

I would like to acknowledge the management of Mar Baselios College of Engineering and Technology, our respected Principal, and the faculty members of the Department of Computer Science and Engineering for providing the necessary facilities, resources, and a supportive academic environment that enabled me to carry out this phase of the project effectively.

George Priji Promod

ABSTRACT

Concretes, in the form of bridges, pavements, and walls, can get cracks due to deterioration over time, creating a hazard to the stability of structures and safety of people. In this study, an automated deep learning method is introduced to detect cracks and measure their severity by applying a convolutional neural network with attention mechanism to analyze cracks in concrete using the SDNET2018 structural defects database. Specifically, an Attention U-Net model that utilizes attention gates in an encoder-decoder structure is used in order to eliminate irrelevant background information and focus on crack-like segments. Pseudo-label generation via Frangi filter is adopted to extract binary mask labels for weakly supervised learning based on Tversky loss to tackle the problem of class imbalance in cracks/non-cracks pixels. The quality of the proposed method is evaluated based on Dice coefficient. After obtaining masks, a pipeline is used to calculate damage parameters by measuring area percent of crack, maximum and average widths, crack length, and number of crack regions through skeletonization and connectivity analysis, and then cracks are classified into one of five categories ranging from none to critical.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

The structural soundness of buildings, bridges, pavements, and other concrete structures is essential for safety reasons. The surface of concrete structures develops cracks due to the fatigue of materials, the environment, temperature changes, and physical load [1]. Upon initial observation, one might think that such cracks are superficial. But when cracks go unchecked and ignored, they tend to become severe and eventually lead to structural failure. As a result, it is imperative that concrete structures undergo surface inspections on a consistent basis [2].

Traditionally, crack detection is carried out by manual visual inspection by trained engineers. This approach is successful in the hands of experts, but has several well-known limitations. It is time-consuming, subjective, inconsistent among inspectors and especially unreliable for detecting hairline cracks on surfaces with complex or varied textures such as pavements and bridge decks. Furthermore, the manual inspection of large structures is physically strenuous, often requiring the use of scaffolding or special equipment, which significantly increases the cost and inspection duration [3].

The recent rapid development in computer vision and deep learning over the last ten years provides a new avenue for automated, objective and scalable structural inspection. Convolutional neural networks (CNNs) have shown great capability to learn discriminative visual features from image data, which makes them a good candidate to identify subtle crack patterns on various surface types. More recently, semantic segmentation architectures have evolved beyond binary classification to pixel-level crack localisation, enabling accurate mapping of crack boundaries within an image [4, 5].

Despite these advances, there remain two major gaps within existing research. Firstly, most crack detection algorithms can simply detect the presence of cracks, and their location. What engineers cannot know is the extent of damage based on such information, such as the dimensions of cracks or the level of danger caused by them. The second issue that arises when discussing deep learning algorithms is that many regard them as black boxes – engineers rarely get any explanation as to why certain parts of the structure were detected as having cracks. The overall task of developing an efficient and explainable automated system, utilizing intelligent optimization and effective classification, explains

the methodology employed here [6].

1.1 Problem Statement

The primary aim of the current research work is to design an automated crack analysis framework for multi-textured concrete surfaces that can do much more than just detecting cracks. The framework should be able to:

- Automatically segment crack regions pixel-wise in diverse surfaces such as walls, road surface, and bridges.
- Evaluate the depth of the crack through its geometrical properties obtained from the segmentation masks generated by the model.
- Provide good justification for the decisions made by the model, so that the engineers can understand and accept the recommendations of the model.
- Apply visualization methods based on gradients in order to identify the factors that contribute to model failures in hard cases.

A considerable challenge in our research is that the primary dataset used for training, SDNET2018, does not have pixel-level segmentation masks. Hence, the system needs to generate simulated ground-truth segmentation masks by using image processing techniques, then use a segmentation model trained on simulated data and a secondary dataset to test both segmentation and severity estimation.

1.2 Objectives

The specific objectives of the proposed research are listed below:

1. To produce pseudo ground truth crack masks based on the SDNET2018 dataset via a classic image processing algorithm consisting of bilateral filtering, CLAHE, morphological blackhat, Frangi vessel enhancement, Canny edge detector, and connected components filter.
2. To develop and train an attention-based U-Net architecture for segmentation tasks with the use of image-mask data pairs obtained from objective 1 above using Tversky Loss and mixed precision training for dealing with class imbalances.

3. To develop a pipeline to analyse crack severity that calculates geometric measures for cracks on an image-by-image basis, including crack area percentage, maximum width, average width, crack length, and number of crack segments, and assigns severity levels to images based on one of five categories: **NONE, LOW, MEDIUM, HIGH, or CRITICAL**.

4. To test the trained model on a separate crack segmentation dataset with ground truth masks to generate IoU and Dice scores per image and severity measures.

5. Grad-CAM (gradient weighted class activation mapping) applied to failure cases (images where IoU is less than 0.3) in order to find the features maps in the encoder responsible for the wrong prediction, as mentioned in the title of the project.

1.3 Scope of the Work

This project is scoped to RGB image-based crack analysis on concrete surfaces. The work covers the complete pipeline from raw image input to severity report output, including pseudo mask generation, deep learning model training, severity analysis, and interpretability. The system operates on still images and does not extend to video-based inspection or real-time mobile deployment, which are identified as future work.

Severity thresholds used in this work are defined based on crack area percentage and are tunable to align with structural engineering standards such as IS 456 or ACI 318. Real-world physical crack width in millimetres is not computed in the current implementation, as it requires known camera calibration parameters. However, the pixel-level width estimates provided can be converted to physical units given such calibration data.

CHAPTER 2

LITERATURE SURVEY

Initially, all the automatic methods used for detecting cracks depended only on conventional image processing tools without any element of learning. These methods operated on the assumption that cracks appear as dark, elongated, low-intensity regions against a lighter background. Thresholding-based methods were among the first to be used. It employed a global or adaptive intensity threshold to separate crack pixels from the background. Thresholding methods were easy to use, but they were very sensitive to uneven lighting,stains on surfaces, and changes in texture.

After that, morphological operations like blackhat transform, dilation, and erosion were used to make crack-like features look better before thresholding. Blackhat Transform was successful in rendering thin dark features prominent over bright background images [7]. Many methods of edge detection were applied, such as Sobel method, Laplacian of Gaussian and Canny Operator. However, many of these methods were sensitive not only to cracks but also to texture boundaries, resulting in large number of false positive [8].

To deal with the thin structure of cracks, vessel enhancement filters like the Frangi filter were adapted from medical imaging. These filters look at the Hessian matrix's eigenvalues at different scales and are better at picking up hairline cracks than other types of filters [9]. Even with these improvements, classical pipelines still need a lot of parameter tuning and have trouble working on surfaces with more than one texture.

2.1 CNN-Based Classification Approaches

Another innovation in crack detection was the adoption of Deep Convolutional Neural Networks (CNN). Unlike feature-based methods that rely on hand-crafted features, the CNN learns discriminative features from the raw input. According to Cha et al. in 2017 [10], a CNN trained on cracks and non-cracks datasets achieves excellent classification accuracy.

The authors Li and Zhao (2019) developed an innovative AlexNet model utilizing a sliding window approach from a massive dataset containing bridge images. Despite the

success of this model, it was computationally expensive since it examined the entire image [11].

GoogLeNet and ResNet18 were employed by Sharma et al. (2020) to study transfer learning in SDNET2018. Improvements in wall and bridge classifications were noted, but minimal improvements in pavement textures were observed [12].

One such algorithm includes the NB-CNN algorithm of Chen and Jahanshahi (2018), which uses the Naive Bayes approach to reduce false positives while localizing the object using Fast R-CNN. Even though these algorithms have been very effective, they fail to capture spatial information [13].

2.2 Segmentation-Based Crack Detection

But to overcome the limitations of classifiers, semantic segmentation techniques have been widely utilized. The Unet architecture created by Ronneberger et al. (2015) is an appropriate model for segmentation because of its encoder-decoder architecture and skip connections [5].

Features are extracted through the encoder whereas the decoder helps in reconstituting the spatial information. Skip connections aid in maintaining the small cracks that are generally ignored because of downsampling

U-Net has shown great potential for crack segmentation on various surfaces according to Golding et al. (2022). [14] Zadeh et al. (2023) have proved the superior performance of encoder-decoder networks than the fully convolutional [15].

One of the problems associated with segmentation is that of unbalanced data since the cracks account for only a tiny fraction of the image. The use of loss functions like Dice loss and Tversky loss has been suggested to help solve this problem. Tversky loss can assign weights to both false negatives and false positives.

2.3 Attention Mechanisms in Segmentation

Standard U-Net architectures treat all spatial regions equally, which can lead to false positives in textured surfaces. Attention mechanisms improve performance by allowing the model to focus on relevant regions.

Attention U-Net, introduced by Oktay et al. (2018), employs attention gates that apply

gating information from deep layers to control skip connections between them. These gates allocate weight to certain spatial locations and eliminate unnecessary background features [16].

For crack detection, the use of attention models enables the distinction between cracks and other textured regions resembling road patterns. Also, attention maps give some level of interpretation to where the model concentrates while making predictions.

2.4 Severity Assessment in Structural Inspection

While there is ample research literature on crack detection and crack segmentation, severity estimation research is comparatively minimal. Structural design standards like IS 456 and ACI 318 have classified crack severity based on quantitative characteristics like crack width [17].

Skeletonization technique can be employed to represent the crack masks in such a way that they become one-pixel wide, making it possible to measure the lengths of cracks. The distances from the skeleton to the edges can give approximations for crack width.

These approaches yield geometrical measurements like crack width, length, and area that may be employed to classify the severity. However, the integration of these approaches into deep learning workflows is still underdeveloped.

2.5 Interpretability in Deep Learning Models

Deep learning algorithms have the major disadvantage of being referred to as black-box approaches. The Grad-CAM method, developed by Selvaraju et al., 2017, has been among the most widely used methods for deep learning explanations. This approach involves computing gradients of the target output with regard to feature maps and then generating heatmaps indicating significant areas affecting predictions [18].

In the case of segmentation, the average of the outputted mask is chosen as the scalar. Thus, regions responsible for the crack prediction can be detected. Although Grad-CAM has been widely applied in medical imaging, the use of this method in crack prediction has not been explored enough yet.

2.6 Research Gap and Motivation for Proposed Work

The literature review reveals several key gaps:

- **Gap 1:** Lack of pixel-level ground truth masks in SDNET2018 requiring pseudo mask generation.
- **Gap 2:** Absence of severity analysis in deep learning pipelines, limiting practical usability.
- **Gap 3:** Lack of interpretability in crack detection systems, reducing trust in predictions.
- **Gap 4:** Limited focus on multi-texture surfaces, affecting generalisation.

These gaps motivate the proposed work, which integrates pseudo mask generation, segmentation using Attention U-Net, severity analysis, and Grad-CAM-based interpretability into a unified framework.

CHAPTER 3

DATASET

The quality and variety of the training data play an essential role in determining the performance of a model in a deep learning-based system. This chapter presents the datasets employed in this research: SDNET2018 dataset [1], which is used for training the models, and the Kaggle crack segmentation dataset, which is available online for quantitative assessment. This chapter also introduces the mask generation process devised due to the lack of annotation in the SDNET2018 dataset.

3.1 SDNET2018 Dataset

3.1.1 Description and Structure

SDNET2018 is a large-scale annotated image dataset for non-contact concrete crack detection and structural health monitoring by deep learning techniques. It was developed by Dorafshan et al. at Utah State University and released to the public to assist in automated infrastructure inspection research. The dataset contains more than 56,000 images of concrete surfaces collected from real-world structures and is one of the largest public datasets for crack detection available at the time of its release.

The images in SDNET2018 are 256×256 pixel RGB images cropped from high resolution images of real structural surfaces. The images are provided with a binary image-level label indicating whether the image contains a crack or not. Cracked images are designated by a C at the beginning of the filename and non-cracked images by a U for easy distinction in programmatic terms.

There are three main categories in the dataset, each of which corresponds to a different type of structural surface: Walls (W), Bridge Decks (D), and Pavements (P). There are both cracked and uncracked subfolders in each category. The bridge deck category has about 26,000 images, the wall category has about 16,000 images, and the pavement category has about 14,000 images. This dataset's ability to cover multiple surfaces is a major strength when it comes to training models that can work with different types of infrastructure.

(a) Wall – Cracked #1 **(b)** Wall – Cracked #2 **(c)** Deck – Cracked #1

(d) Deck – Cracked #2 **(e)** Pavement – Cracked #1 **(f)** Pavement – Cracked #2

Figure 3.1: Sample cracked surface images from the SDNET2018 dataset. The three structural surface categories are Walls (a–b), Bridge Decks (c–d), and Pavements (e–f). Each image is a 256×256 pixel RGB crop annotated at the image level as cracked.

The cracks present in the data set range from extremely small, fine cracks, of 0.06 mm width, to larger structural cracks which can be as wide as 25 mm. There is, therefore, considerable variation in crack severity that will be used to train the model. It should also be noted that the textures on the surfaces of all the three materials differ greatly. Wall surfaces tend to have a smooth appearance while bridge deck surfaces are quite grainy. Pavement surfaces, however, are the most complicated.

3.1.2 Limitations — No Pixel-Level Ground Truth Masks

The main limitation of SDNET2018 for this project is that it offers only image-level binary labels. It does not include pixel-level segmentation masks that show which pixels in each image belong to crack areas. This issue is common in large crack datasets because manually annotating pixel-level masks for tens of thousands of images requires a lot of effort.

This limitation means that SDNET2018 cannot be used directly to train a semantic segmentation model like U-Net, which requires paired image and mask data for supervised training. To address this issue, we developed a pseudo ground truth mask generation

pipeline using traditional image processing techniques. The created pseudo masks act as approximate pixel-level supervision labels for training the Attention U-Net segmentation model. You can find the details of this pipeline in Section 3.3.

3.2 Crack Segmentation Dataset (Kaggle)

3.2.1 Description and Structure

The second dataset for this project is the crack segmentation dataset that Aravind Nagarajan put on Kaggle (aravindnagarajan/crack-segmentation-dataset). This dataset has real pixel-level ground truth segmentation masks for each image which makes it better for using standard metrics like Intersection over Union (IoU) and Dice coefficient to quantitatively evaluate the trained segmentation model.

The dataset is organized so that there is a clear split between the train and test sets. Each split has two subfolders that is an images folder with the original RGB crack images in JPEG format and a masks folder with binary segmentation masks that show where the cracks are (white pixels) and where the background is (black pixels). Each mask image has the same name as the input image it goes with which makes it easy to match them up when testing.

The pictures in this dataset show concrete surfaces with cracks that are different widths and depths. The textures on the surfaces are very similar to those on the walls and pavements in SDNET2018 which makes them a good place to test the model that was trained on pseudo masks from SDNET2018.

3.2.2 Role in this Project

In this work, this dataset has two main uses. First, it is used as the test set for quantitative segmentation evaluation. The trained Attention U-Net model runs in inference mode on the test split images, and the predicted masks are compared to the true ground truth masks to get per-image IoU and Dice scores. This gives us an objective way to see how well the model trained on pseudo-masks works on images with real pixel-level annotations.

Second, this dataset is used as the starting point for the severity analysis pipeline. For each test image, the predicted binary mask is passed through the severity analysis module to compute crack area percentage, maximum and mean crack width, crack length, and

number of distinct crack regions. The resulting severity level and geometric metrics are saved to a CSV report alongside the IoU and Dice scores, enabling joint analysis of segmentation quality and severity estimation.

It is important to use a separate evaluation dataset with real ground truth masks. The Attention U-Net was trained on pseudo masks created by classical image processing from SDNET2018. Evaluating it on the same data would not provide a reliable measure of generalization. The Kaggle crack segmentation dataset has human-annotated masks that can be used as a separate test bed. This makes the evaluation more reliable and the IoU/Dice numbers more useful.

3.3 Pseudo Ground Truth Mask Generation

3.3.1 Why Pseudo Masks Were Needed

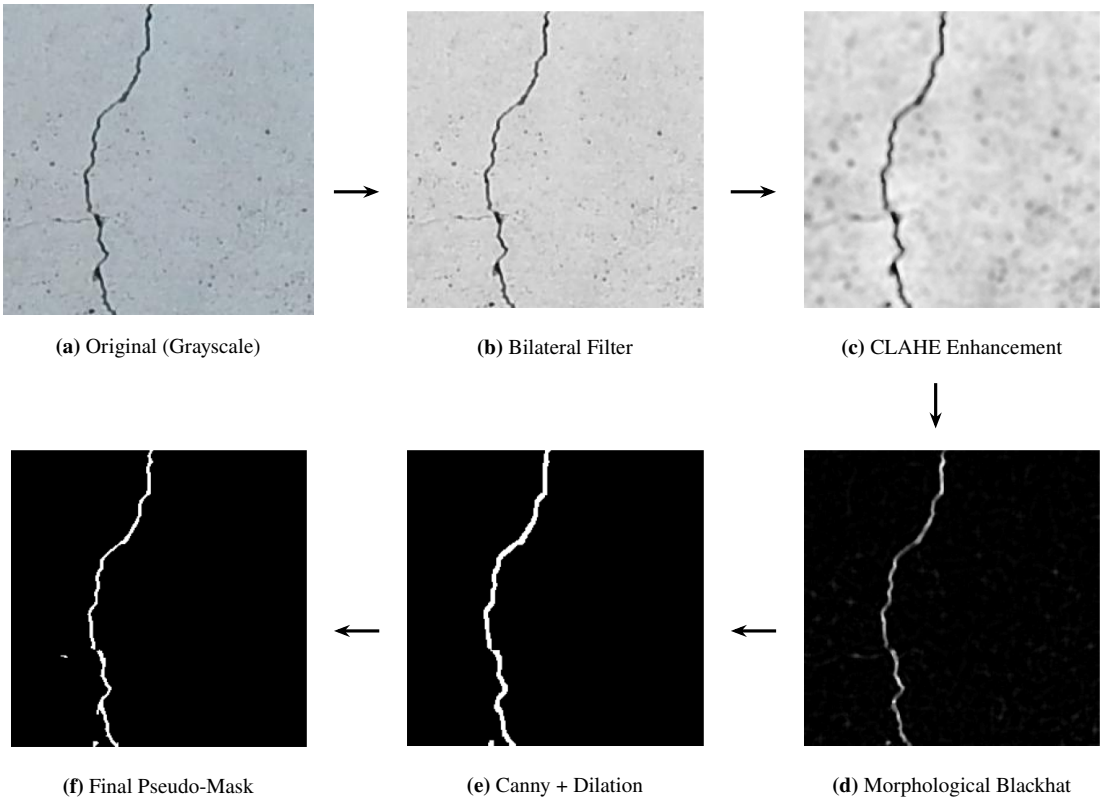
As explained in Section 3.1.2, SDNET2018 only gives image-level labels and not pixel-level crack annotations. To train a segmentation model, you need pairs of images and masks, with each pixel in the mask labeled as either "crack" or "background." To close this gap, a pseudo ground truth mask generation pipeline was made that uses traditional image processing methods to automatically make binary crack masks from the raw SDNET2018 images.

This method, often referred to as weakly supervised or pseudo-label training, acknowledges that the generated masks will not be 100% accurate. The process may miss certain small cracks and may also include some false positive regions. However, the objective is to produce masks that are sufficiently accurate to provide useful pixel-level supervision for the segmentation model. It is expected that the deep learning model will learn to generalize beyond the noise present in the pseudo labels, particularly when trained using a robust loss function such as Tversky loss, which is capable of handling label noise and class imbalance.

To show the model that some images don't have any crack pixels, a fully black mask of all zeros was made for non-cracked images from SDNET2018. This is important so that the model doesn't make up crack predictions on clean surfaces.

3.3.2 OpenCV Pipeline

The pseudo mask generation pipeline processes each cracked image through the following sequence of operations:



(a) Original (Grayscale) **(b)** Bilateral Filter **(c)** CLAHE Enhancement **(f)** Final Pseudo-Mask **(e)** Canny + Dilation **(d)** Morphological Blackhat

Figure 3.2: Step-by-step visualisation of the OpenCV pseudo ground truth mask generation pipeline applied to a cracked wall image from SDNET2018. The processing stages are Grayscale conversion (a), Bilateral filtering (b), CLAHE contrast enhancement (c), Morphological blackhat transform (d), Canny edge detection with dilation (e), and Morphological closing with connected component filtering (f).

1. Bilateral Filtering: The image is converted to a grayscale image and then filtered by bilateral filtering. Bilateral filtering smooths out the surfaces without removing the edges unlike Gaussian smoothing. It helps reduce the noise in terms of textures of the surface while not removing any cracks that might be on the surface [19].

2. CLAHE Contrast Enhancement: Contrast Limited Adaptive Histogram Equalisation (CLAHE) is employed for improving the contrast locally in the filtered image. CLAHE technique involves segmentation of an image into smaller blocks and applying histogram equalization separately on each block. This procedure helps in identifying low-contrast crack regions which are not visible under uniform lighting conditions [20].

3. Morphological Blackhat Transform: Morphological Blackhat Transform is performed through the use of the square shaped structural element. Blackhat transform calculates the subtraction between morphological closing of the image and the original image, thereby emphasizing the presence of dark areas whose dimensions are smaller than the structural element used for the transform. Given that cracks have a dark appearance on a relatively light background, the blackhat transform will yield high values at the position of cracks.

4. Canny Edge Detection: The process of edge detection based on the Canny method is then employed on the black hat enhanced image to obtain the edges along the cracks. The Canny method employs a hysteresis scheme using dual thresholds to detect edge pixels of high and low strength, linking low edge pixels to high edge pixels if they are neighboring.

5. Dilation and Morphological Closing: The binary edge map generated by the Canny operator undergoes dilation to close gaps among neighboring edge points before performing morphological closing to complete the fractured crack regions. This process is essential since thin cracks will generate segmented edges, and through closing, all the segmented cracks will be united into one region in the final crack map.

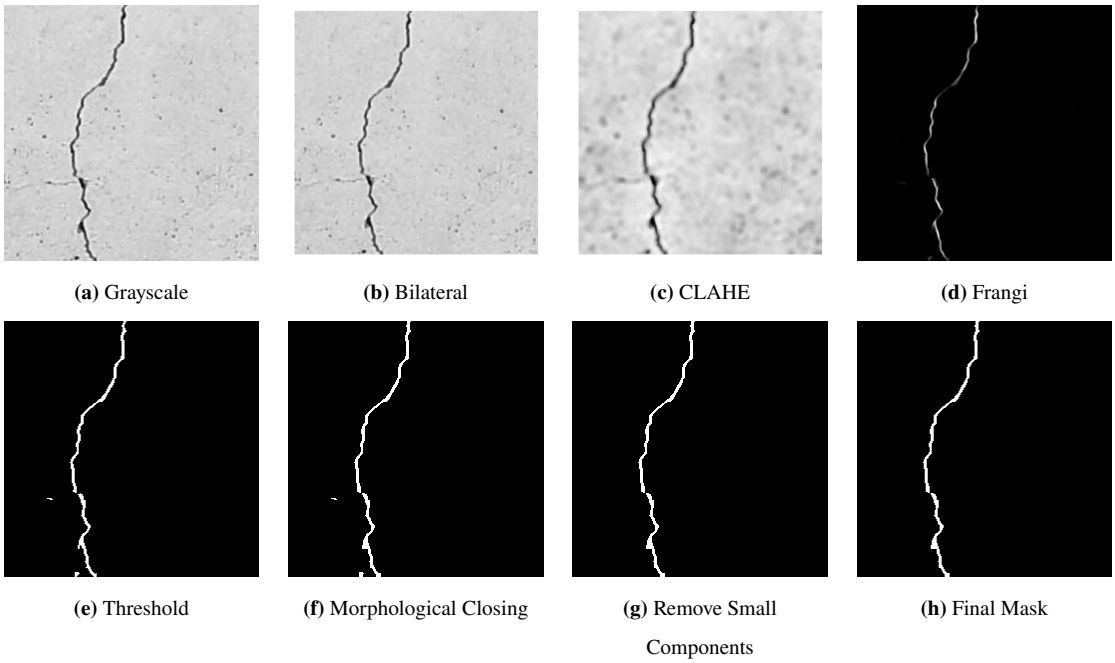
6. Connected Component Filtering: However, the generated binary mask not only comprises the crack features but also includes some noisy regions within itself. For this reason, connected component labeling is performed in order to identify all unique regions of the mask. The components having areas less than a certain minimum value are filtered out in order to reduce noise in the generated mask.

3.3.3 Frangi Vessel Enhancement Filter

In addition to the OpenCV pipeline, the Frangi vessel enhancement filter was incorporated into the pseudo mask generation process. The Frangi filter originally developed for enhancing blood vessels in medical images and is operated by analysing the eigenvalues of the Hessian matrix of the image intensity function at multiple scales. At each pixel, the Hessian matrix captures the local curvature of the intensity surface in all directions. For a tubular structure such as a crack, the Hessian has one eigenvalue close to zero along the crack direction and one large negative eigenvalue perpendicular to it. The Frangi filter combines these eigenvalues into a vesselness measure that is high at crack-like elongated dark structures and low elsewhere.

The multi-scale nature of the Frangi filter is particularly important for crack detection

as cracks vary considerably in width across a single image and across different images. The filter can find both hairline cracks and larger structural cracks with the same level of sensitivity by calculating the vesselness measure at different scales and taking the highest response across all scales. To make the final pseudo mask, we used a logical OR operation to combine the output of the OpenCV pipeline with the response of the Frangi filter. This made sure that both Frangi and Canny found thin cracks that the other did not.



(a) Grayscale **(b)** Bilateral **(c)** CLAHE **(d)** Frangi

(e) Threshold **(f)** Morphological Closing **(g)** Remove Small Components **(h)** Final Mask

Figure 3.3: Step-by-step visualisation of the proposed crack detection pipeline using Frangi filtering and morphological refinement applied to a cracked surface image. The processing stages are Grayscale conversion (a), Bilateral filtering (b), CLAHE contrast enhancement (c), Frangi filter enhancement (d), Adaptive thresholding (e), Morphological closing (f), Connected component analysis (g), and Final refined binary mask (h).

3.3.4 Dataset Preparation for Training

After pseudo mask generation, the dataset was structured as image-mask pairs with a defined train/validation split. Images and their corresponding masks were resized to 448×448 pixels to match the input resolution of the Attention U-Net model. This higher resolution compared to the original 256×256 pixels of SDNET2018 was chosen to preserve fine crack detail, and was made feasible by the use of mixed precision training which reduced GPU memory consumption by approximately 40

Both cracked images with generated pseudo masks and non-cracked images with fully black masks were included in the training set. The inclusion of non-cracked images is important because it teaches the model that the correct output for a clean surface

is an all-zero mask, which prevents the model from over-predicting crack regions on non-cracked test images. The final dataset used for training had a good mix of cracked and non-cracked images from all three surface types in SDNET2018: walls, bridge decks, and pavements. This way, the model was trained on all kinds of surface texture conditions.

The above sections have covered the datasets SDNET2018 training dataset [1] and Kaggle crack segmentation evaluation dataset. We have also discussed the rationale behind generating pseudo masks and provided a comprehensive overview of the processes involved in generating pseudo masks using the OpenCV and Frangi filters. Chapter 4 will cover the Attention U-Net model and its training strategy.

CHAPTER 4

METHODOLOGY

This chapter presents the complete methodology of the proposed crack analysis framework, covering image preprocessing, the Attention U-Net segmentation architecture [16], the training configuration, the crack severity analysis pipeline, and the Grad-CAM interpretability module [18]. Each design decision is motivated by the requirements identified in the literature survey and the constraints imposed by the available data.

4.1 System Overview and Architecture

The proposed system is an end-to-end crack analysis pipeline that takes a raw RGB image of a concrete surface as input and produces three categories of output: a pixel-level binary crack mask, a quantitative severity report, and a Grad-CAM interpretability heatmap for failure case analysis. The pipeline is divided into four major stages that execute sequentially — preprocessing, segmentation, severity analysis, and interpretability.

During stage one, the image undergoes resizing to 448x448 pixel dimensions and normalization to a range between [0, 1]. During the second stage, the pre-processed image is fed through the learned Attention U-Net architecture to predict a probability map at the same spatial resolution. Thresholding of this probability map with 0.5 results in the binary crack mask prediction. The binary mask prediction is further processed using geometric analysis by skeletonization and distance transformations to obtain crack metrics and a corresponding severity level. Grad-CAM is selectively applied to images that have a poor prediction match between the prediction mask and ground-truth in the fourth stage, resulting in a heat map of encoder features used for making predictions. The full system flow is depicted in the description of each individual stage below.

4.2 Image Preprocessing Pipeline

All input images are subjected to a consistent preprocessing pipeline before being passed to the model. Each image is read using OpenCV in BGR format and converted to RGB.

The image is resized to 448×448 pixels using bilinear interpolation, which preserves fine crack detail better than nearest-neighbour interpolation at this resolution. Pixel values are then normalised by dividing by 255.0 to bring them into the floating-point range [0, 1], which is the expected input range for the model.

During training, the same preprocessing steps are applied to both the image and its corresponding pseudo mask. Mask images are resized using nearest-neighbour interpolation to avoid introducing intermediate pixel values at crack boundaries, and binarised at a threshold of 127 to produce clean binary labels with values of exactly 0 or 255.

No data augmentation was applied in the current implementation. The large size and diversity of SDNET2018 that covers three surface types with over 56,000 images which was considered sufficient to provide the model with adequate variation during training without the need for synthetic augmentation. This is consistent with several works in the crack segmentation literature that have found diminishing returns from augmentation when the base dataset is already diverse in texture and illumination conditions.

4.3 Attention U-Net Architecture

4.3.1 Overall Structure

The Attention U-Net is the segmentation model used in this work. It adds attention gate mechanisms to each skip connection in the standard U-Net architecture. The model follows the classic encoder-decoder structure with four levels of encoding, a bottleneck, and four corresponding levels of decoding. The filter progression across levels is $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ at the encoder, with a bottleneck of 512 filters, and $256 \rightarrow 128 \rightarrow 64 \rightarrow 32$ at the decoder. The last layer of output is a 1×1 convolution with a sigmoid activation that makes a single-channel probability map with the same spatial resolution as the input.

The input shape is 448×448×3, representing an RGB image at the resolution selected for training. The output shape is 448×448×1, where each value indicates the model's predicted probability that the corresponding pixel belongs to a crack region. The total parameter count of the architecture is approximately 7.76 million.

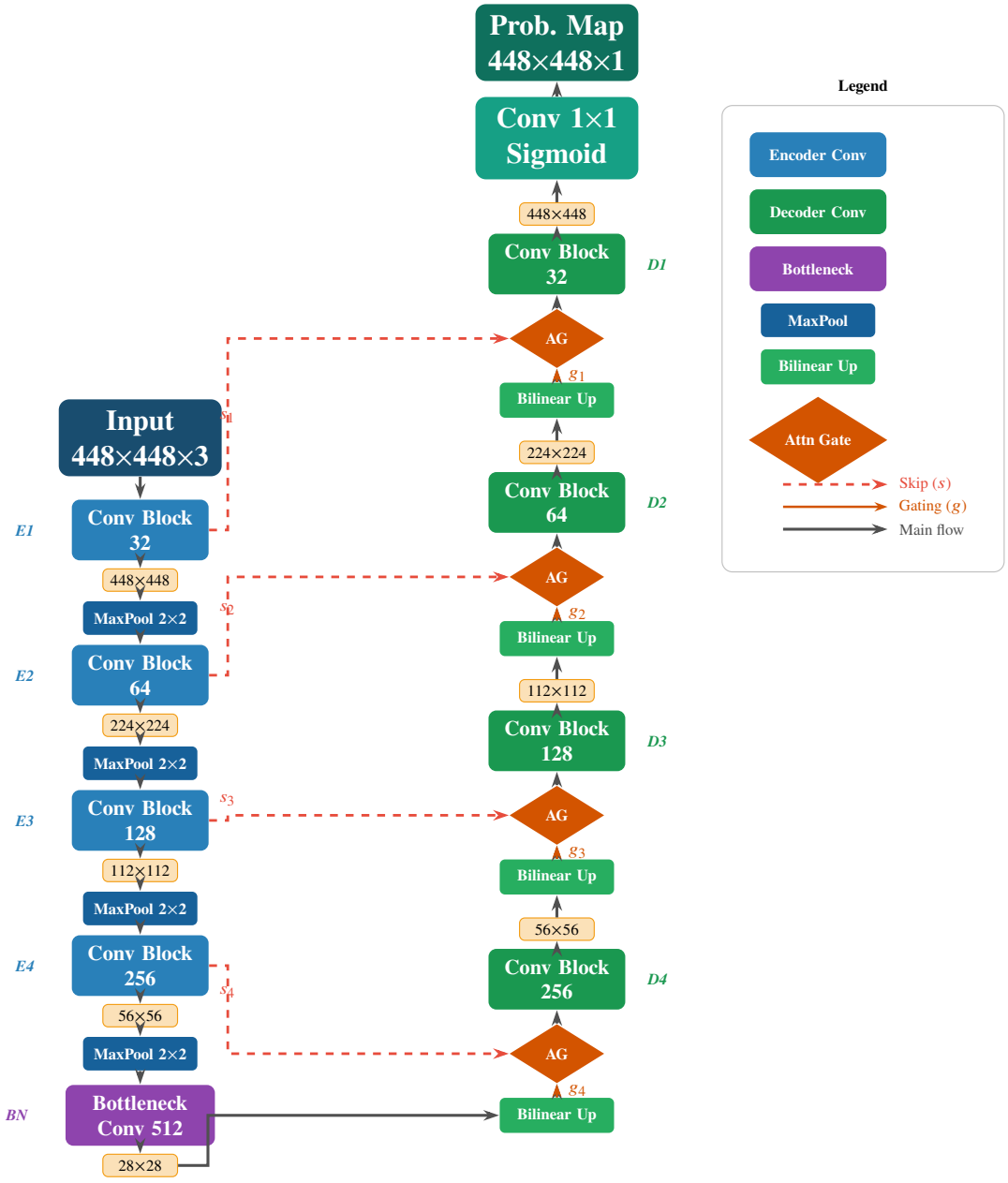


Figure 4.1: Architecture of the Attention U-Net used for crack segmentation

4.3.2 Encoder Path

Each encoder level consists of a conv block followed by a max pooling operation. The conv block applies two consecutive sequences of Conv2D (3×3, same padding) → Batch Normalisation → ReLU activation. The use of batch normalisation after each convolution stabilises training by normalising the distribution of activations, which is particularly important given the class imbalance between crack and background pixels. Max pooling with a 2×2 kernel and stride 2 halves the spatial resolution at each level while doubling the receptive field.

The skip connection output from each encoder level — taken before pooling — is stored for later use in the corresponding decoder level. These skip connections carry high-resolution spatial detail that would otherwise be lost during downsampling, and are critical for reconstructing precise crack boundaries in the decoder.

4.3.3 Bottleneck

The bottleneck is a single convolutional block with 512 filters applied to the feature map that has been downsampled the most. The input has a spatial resolution of 28×28 for the 448×448 input. Each feature map unit's receptive field covers a large part of the input image at this level. This lets the model get a sense of the overall layout of the crack before the decoder starts to put the pieces back together.

4.3.4 Decoder Path

Each decoder level begins with bilinear upsampling that doubles the spatial resolution of the incoming feature map. The upsampled feature map is then passed through an attention gate together with the corresponding skip connection from the encoder, producing an attention-weighted version of the skip connection. The attention-weighted skip connection is concatenated with the upsampled feature map and passed through a conv block identical in structure to those in the encoder. This process is repeated at all four decoder levels until the original spatial resolution of 448×448 is restored.

Bilinear upsampling was chosen over transposed convolution for the decoder upsampling step. Transposed convolution can introduce checkerboard artefacts in the output probability map, which can cause fragmented crack predictions. Bilinear upsampling followed by convolution produces smoother probability maps and was found to give more continuous crack mask predictions during initial experiments.

4.3.5 Attention Gate Mechanism

The attention gate is the key architectural difference between standard U-Net and the Attention U-Net used in this work. At each decoder level, an attention gate takes two inputs — the gating signal g, which is the upsampled feature map from the decoder, and the skip connection signal s from the corresponding encoder level. Both g and s are passed through separate 1×1 convolutions that project them to a common number of filters, followed by batch normalisation. The two projections are added element-wise and passed through a ReLU activation. A final 1×1 convolution with sigmoid activation produces a spatial attention map with values between 0 and 1, one per spatial location.

This attention map is multiplied element-wise with the original skip connection s, producing an attention-weighted skip connection where spatial locations relevant to crack prediction are preserved and irrelevant background regions are suppressed. The decoder's gating signal is a top-down contextual signal that tells the attention gate to pay attention to parts of the encoder features that are relevant to the crack.

For crack segmentation on multi-texture surfaces, the attention gate provides a significant practical benefit. Surface stains, aggregate boundaries, and joint lines in pavement and bridge deck images produce strong responses in the encoder that resemble crack features. Without attention, these spurious encoder activations would be passed directly to the decoder via skip connections, causing false positive crack predictions. The attention gate suppresses these responses by using the decoder's higher-level understanding of crack context to modulate which encoder activations are passed forward.

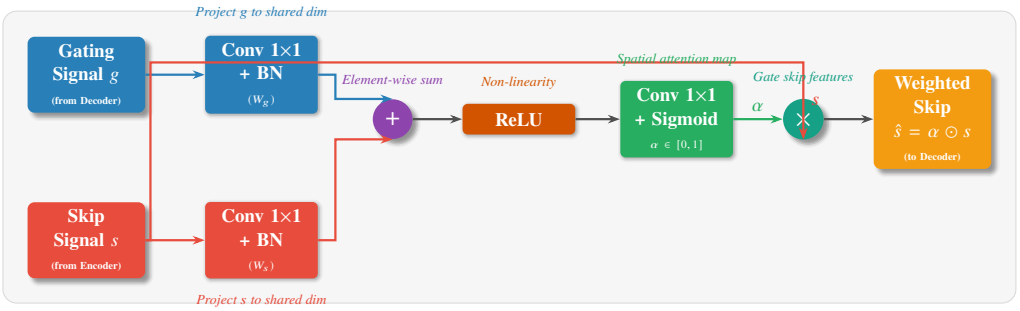


Figure 4.2: Detailed diagram of the Attention Gate mechanism

For crack segmentation on multi-texture surfaces, the attention gate provides a significant practical benefit. Surface stains, aggregate boundaries, and joint lines in pavement and bridge deck images produce strong responses in the encoder that resemble crack features. Without attention, these spurious encoder activations would be passed directly to the decoder via skip connections, causing false positive crack predictions. The attention gate suppresses these responses by using the decoder's higher-level understanding of

crack context to modulate which encoder activations are passed forward.

4.3.6 Output Layer

The final layer of the decoder is a Conv2D layer with a single filter, 1×1 kernel, same padding and sigmoid activation. The sigmoid activation maps the output to the range [0, 1] at each pixel and producing a probability map that can be thresholded to obtain the binary crack mask. The output layer uses float32 precision regardless of the mixed precision training configuration, ensuring numerical stability in the sigmoid computation.

4.4 Training Configuration

4.4.1 Loss Function — Tversky Loss

The Tversky loss function was used to train the model. Tversky loss is a generalisation of Dice loss that introduces two hyperparameters, alpha and beta, which independently control the penalty applied to false positives and false negatives respectively. The Tversky index is defined as:

$$TI = \frac{TP}{TP + \alpha \cdot FP + \beta \cdot FN} \tag{4.1}$$

where TP, FP, and FN denote the number of true positive, false positive, and false negative crack pixels respectively. The Tversky loss is then defined as $1 - TI$. When alpha = beta = 0.5, the Tversky loss reduces exactly to the standard Dice loss.

In the present work, alpha was set to 0.3 and beta was set to 0.7, placing a higher penalty on false negatives than on false positives. The use of such an asymmetrical weighing factor is justified in this case because the absence of a crack in a structural inspection would be much more hazardous than a false detection. This is due to the high value of beta, which makes the algorithm less conservative and prone to detecting cracks in pixels that might not have them.

The Standard Binary Cross Entropy loss, utilized during the initial training phase of this project, was observed to heavily bias towards predicting the background due to an extremely uneven distribution between crack and background classes. The Dice loss mitigated this problem; however, it considered false positives and false negatives on

equal footing. The optimal performance in terms of Dice scores during preliminary testing was achieved by the Tversky loss with the proposed asymmetric weight.

4.4.2 Evaluation Metric — Dice Coefficient

The Dice coefficient was used as the primary evaluation metric during training and validation. The Dice coefficient is defined as:

$$Dice = \frac{2 \cdot |P \cap G|}{|P| + |G|} \tag{4.2}$$

where P is the set of predicted crack pixels and G is the set of ground truth crack pixels. The Dice coefficient ranges from 0 (no overlap) to 1 (perfect overlap) and is robust to class imbalance because it measures the overlap relative to the size of the predicted and ground truth regions rather than the total number of pixels.

During training, the Dice coefficient was monitored on the validation set after each epoch. The model checkpoint with the highest validation Dice across all training epochs was saved and used for all subsequent inference and evaluation.

4.4.3 Mixed Precision Training

Mixed precision training was employed using float16 computation with float32 output casting to reduce GPU memory consumption during training. In this approach, the forward pass and gradient calculations are performed in float16, which reduces the memory required for activations and gradients by approximately half compared to standard float32 training. However, the model weights are maintained in float32 to ensure numerical stability during weight updates. Additionally, before the sigmoid activation, the final output layer is explicitly cast to float32 to prevent instability in the loss computation.

The use of mixed precision resulted in an approximate 40% reduction in GPU memory usage. This enabled training at a higher input resolution of 448×448 pixels, compared to the 256×256 resolution used in the initial stage of the project. The increased resolution improved the model's ability to detect thin hairline cracks, which occupy very few pixels at lower resolutions but become more distinguishable at higher resolutions.

4.4.4 Learning Rate Scheduling

Training was conducted with an initial learning rate of 1×10^{-4} using the Adam optimiser. A learning rate reduction callback was applied to halve the learning rate to 5×10^{-5} when the validation Dice failed to improve for a specified number of consecutive epochs. This schedule was found to stabilise the validation Dice oscillation observed in earlier training runs where a fixed learning rate caused the validation metric to fluctuate by ±0.02 across epochs. The learning rate reduction produced a more stable convergence in the later training epochs and contributed to achieving the best validation Dice of 0.6915.

4.4.5 Early Stopping

An early stopping callback was used to terminate training when the validation loss failed to improve for a patience period of several epochs with automatic restoration of the best model weights at the end of training. This stopped overfitting in the later epochs when the training Dice kept getting better but the validation Dice stayed the same. Training reached convergence at 30 epochs without activating the early stopping criterion signifying that the model had not commenced overfitting within the training budget.

4.5 Crack Severity Analysis Pipeline

4.5.1 Overview

The severity analysis pipeline operates on the binary predicted crack mask produced by the Attention U-Net model. It extracts five geometric properties from the mask — crack area percentage, maximum crack width, mean crack width, crack length and number of distinct crack regions and uses these to classify each image into one of five severity levels. The pipeline is implemented in the compute severity function and is applied to every image in the test set during batch processing.

4.5.2 Crack Area Percentage

The crack area percentage is computed as the ratio of crack pixels to total image pixels, expressed as a percentage:

$$\text{Crack Area } \% = \frac{\text{Number of crack pixels}}{\text{Total pixels}} \times 100 \tag{4.3}$$

For a 448×448 image, the total pixel count is 200,704. The crack area percentage provides a normalised measure of crack density that is independent of image resolution and is directly comparable across images of different physical scales.

4.5.3 Skeletonization for Crack Length and Width

Using skeletonization on the binary crack mask, we can find the centerline of the crack, which is one pixel wide. The skeletonization algorithm repeatedly takes away boundary pixels from the crack area while keeping connectivity and moving toward a topological skeleton that shows the medial axis of each crack.The total number of pixels in the skeleton provides the crack length estimate in pixels.

To estimate crack width, the Euclidean distance transform of the binary crack mask is computed. The distance transform assigns to each crack pixel its distance to the nearest background pixel. Evaluating the distance transform at the skeleton pixels gives the local half-width of the crack at each point along its centerline. The maximum crack width and mean crack width are then computed as twice the maximum and mean of these skeleton-evaluated distance values respectively and corresponding to the full crack width at each measured point.

This method gives a physically meaningful measure of crack width that takes into account the actual shape of the crack instead of just the bounding box. It works well with tapered cracks, where the width changes along the length of the crack and branching cracks, where two or more crack arms meet at a junction point.

4.5.4 Number of Crack Regions

Connected component labelling is performed on the crack mask in order to detect discrete crack regions. Each of the connected components corresponds to a separate region of a single crack or group of cracks that are not connected spatially to any other cracks in the image. The total number of connected components serves as an indication of crack fragmentation where a single, contiguous crack will give rise to only one region, whereas several isolated cracks will lead to more regions.

4.5.5 Five-Level Severity Classification

The five severity levels and their corresponding crack area percentage thresholds are defined as follows:

Severity Level	**Crack Area Threshold**	**Description**
NONE	0%	No crack pixels detected
LOW	0.01% – 1%	Minor surface cracking, monitoring recommended
MEDIUM	1% – 3%	Moderate cracking, inspection required
HIGH	3% – 7%	Significant cracking, repair recommended
CRITICAL	> 7%	Severe cracking, immediate action required

Table 4.1: Crack Severity Classification Based on Area Percentage

The primary classification criterion is crack area percentage, as it is the most directly measurable and reproducible geometric property. The width and length metrics provide supplementary information in the CSV report that engineers can use to refine the severity assessment based on structural engineering standards applicable to the specific structure type.

4.5.6 Batch Processing and CSV Report

The batch processing pipeline iterates over all images in the test set, applies the inference and severity analysis pipeline to each image, and accumulates the results into a list of dictionaries. Upon completion, the results are written to a CSV file with one row per image, containing the following fields: *filename*, *is_noncracked_gt*, *severity_level*, *crack_area_pct*, *max_width_px*, *mean_width_px*, *crack_length_px*, *num_crack_regions*, *iou*, and *dice*.

For those images in which a mask exists for ground truth in the test masks folder, the IoU and Dice metrics between the predicted mask and the ground truth mask will be calculated and recorded in the CSV file. This allows both the performance of the segmentation algorithm and the severity score to be analyzed together, making it possible to pinpoint those instances in which the model outputs a severe prediction when its IoU is low, thus meaning that it was able to recognize crack-like objects that weren't actually cracks.

Severity overlay visualisations are optionally saved for each test image, showing the

original image, the predicted mask, and a colour-coded overlay of the crack region and skeleton. These visualisations provide a qualitative complement to the quantitative CSV metrics.

4.6 Grad-CAM Interpretability

4.6.1 Overview and Motivation

Grad-CAM (Gradient-weighted Class Activation Mapping) is applied in this work as a post-hoc interpretability method to generate visual explanations for the model's segmentation decisions. As discussed in Chapter 2, the primary motivation for Grad-CAM in this project is not to improve detection performance — the predicted mask already provides pixel-level crack localisation — but to explain why the model makes incorrect predictions on difficult cases. This addresses the interpretability objective stated in the project title and provides diagnostic insight that can inform future model improvements.

4.6.2 Adaptation for Segmentation

Grad-CAM was originally designed for classification models where the target scalar is a class confidence score. For a segmentation model there is no single class score. The standard adaptation for segmentation is to use the mean of the predicted output mask as the target scalar:

$$\text{Loss} = \text{mean}\big(\text{sigmoid}(\text{output})\big) \tag{4.4}$$

The above-mentioned scalar indicates the confidence of the whole model towards predicting crack pixels throughout the whole image. Computing the gradient with respect to the target convolutional layer feature maps via backpropagation of the above-mentioned scalar will result in gradient information indicating the spatial areas of the feature map that most affected the whole crack prediction process. It is a common practice in numerous interpretability works related to segmentation problems.

4.6.3 Target Layer Selection

The selected layer for applying Grad-CAM is the bottleneck conv layer, which is the most deep conv layer in the encoder and acts on the 28×28 feature map with 512 filters. This is

a conscious decision made after balancing between the amount of spatial information and semantic abstraction contained in the layer. Deep layers contain more semantically rich representations of crack-related features with low spatial resolution, whereas shallow layers contain high spatial resolution with little semantic information. In the bottleneck layer, we have the right balance, as it is at a sufficiently coarse resolution to filter out the fine texture noise while still maintaining some spatial information to tell us which part of the image is responsible for making the prediction.

Another useful function implemented is the automatic selection of the bridge conv layer by simply selecting the conv layer at the center of the model depth, which acts as a good alternative if the layer name is not known.

4.6.4 Heatmap Generation

The Grad-CAM heatmap is generated through the following steps. A sub-model is constructed that takes the same input as the full model but produces two outputs simultaneously — the feature maps of the target bottleneck layer and the final predicted mask. A GradientTape context is used to record the forward pass through this sub-model. The target scalar loss — mean of the predicted mask — is computed, and the gradient of this loss with respect to the bottleneck feature maps is computed by calling the tape.

The gradients are globally average-pooled over the spatial dimensions to produce one importance weight per feature map channel. These weights reflect the contribution of each channel to the overall crack prediction. The importance-weighted sum of the feature maps is then computed, producing a single-channel heatmap at the bottleneck resolution of 28×28. A ReLU operation is applied to retain only positive activations — regions that contributed positively to crack prediction — and the heatmap is normalised to the range [0, 1].

The 28×28 heatmap is then upsampled to 448×448 using bilinear interpolation to match the input image resolution. The Jet colormap is applied to convert the scalar heatmap to an RGB colour image, where warm colours (red, orange) indicate high activation — regions the model strongly attended to — and cool colours (blue) indicate low activation. The coloured heatmap is blended with the original image at a 50:50 ratio to produce the final overlay visualisation.

4.6.5 Application on Failure Cases

The technique is used specifically on test images which have an IoU less than 0.3 when compared to the ground truth masks. This means that in such cases, the model has failed to predict the cracks with decent spatial accuracy; either because it detected false positives or due to missing the majority of the cracks.

For each such failure case, a four-panel visualisation is generated showing the original image, the ground truth mask, the predicted mask, and the Grad-CAM overlay. This side-by-side presentation makes it possible to directly compare where the model predicted cracks, where the true cracks are, and what region of the image the model's encoder was attending to when making its prediction. In cases of false positive prediction, the Grad-CAM heatmap typically reveals that the encoder was attending to surface stains, aggregate boundaries, or joint lines that share visual similarity with crack structures. When there is a false negative prediction, the heatmap depicts a more uniform activation pattern, where the focus is less intense than it would be when predicting the presence of a crack, implying that the crack had similar characteristics to those of the surrounding environment.

The methodology for the proposed framework has been explained in this chapter, which includes the image pre-processing process, the model architecture of Attention U-Net with attention gates [16], Tversky loss optimization strategy [21], the mixed precision optimization technique [22], the five level geometric severity detection process, and the model interpretability through Grad-CAM [18]. The experimental findings obtained through this framework will be discussed in chapter 5.

CHAPTER 5

RESULTS AND DISCUSSION

Performance of our framework is addressed in this chapter from two aspects that is quantitative data and qualitative description, with respect to four dimensions: performance during training, segmentation results for the test images from Kaggle database, severity assessment for the whole test distribution and Grad-CAM interpretation for a selected subset. Each result is analyzed with regard to the objectives introduced in Chapter 1 and method described in Chapter 4. This chapter is organized in such a way that each section corresponds to one objective of the project, thus making it easy to assess performance with regard to objectives.

5.1 Training Performance

The Attention U-Net model was trained on Google Colab using a GPU runtime with mixed precision (`float16`) enabled. The training dataset consisted of image–mask pairs derived from the SDNET2018 dataset, with pseudo ground truth masks generated using the OpenCV and Frangi filter pipeline described in Chapter 3.

The model was trained for 30 epochs using the Adam optimiser, with an initial learning rate of 1×10^{-4}, which was reduced to 5×10^{-5} using a learning rate scheduler. Tversky loss was used as the training objective. The best model weights were saved based on the highest validation Dice coefficient observed across all epochs.

5.1.1 Dice Coefficient Progression

The Dice coefficient was monitored on both the training set and the validation set throughout the 30 training epochs. At the beginning of training, the training Dice was approximately 0.37 and the validation Dice was approximately 0.43, indicating that the model had not yet learned meaningful crack representations. Both metrics improved steadily across the first 10 epochs as the model learned to distinguish crack pixels from background texture.

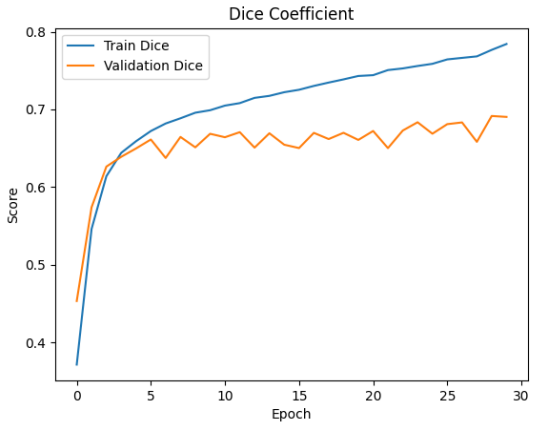


Figure 5.1: Dice coefficient. The Dice coefficient shows a steady increase for both training and validation sets, indicating effective learning of crack features. The validation Dice stabilises around 0.68–0.70, suggesting good generalisation.

The Dice validation showed some fluctuations of about ±0.02 during epochs while training, which is typical for models trained on pseudo-ground truth masks. This is due to the fact that the pseudo masks are not perfect – there are always some cracks that are missing, along with some noise regions that are detected but should be excluded from the final segmentation mask. Thus, the learning signal for the model is inconsistent between examples, leading to a fluctuating validation score instead of gradual convergence.

5.1.2 Loss Progression

The Tversky Loss was consistently reduced from a starting value of about 0.54 for epoch 1 to a training loss of 0.1722 by epoch 30. Likewise, the validation loss showed a consistent decline from about 0.45 to the final value of 0.2702. The difference between the training loss and the validation loss shows that there was slight overfitting as the training proceeded from epochs 16 to 30.

This train-validation gap of approximately 0.098 in the loss and approximately 0.09 in the Dice coefficient is consistent with what is expected when training on pseudo labels. The model inevitably learns some noise patterns specific to the pseudo mask generation pipeline that do not generalise to unseen images. The attention gate mechanism helps mitigate this by suppressing spurious encoder activations, but does not fully eliminate the overfitting effect.

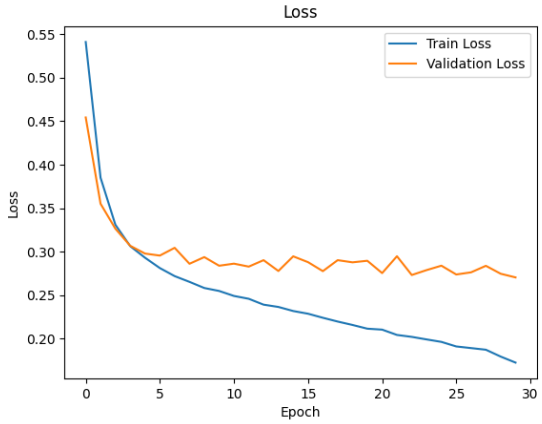


Figure 5.2: Training and validation loss. The loss curves decrease consistently, with training loss continuing to drop while validation loss stabilises after early epochs, indicating mild overfitting but overall stable convergence.

5.2 Segmentation Results on Test Set

For determining the results for the Attention U-Net model, the Attention U-Net model was evaluated on the test data of the crack segmentation dataset available on Kaggle where ground truth masks are available for all the images. Both IoU and Dice scores were determined by computing the similarity between the binary mask obtained from the model to the ground truth mask for every image with the threshold value 0.5.

The mean IoU and mean Dice scores across the test set, along with the distribution of per-image scores and are recorded in the CSV output of the batch severity analysis pipeline. The results demonstrate that the model generalises reasonably well to images with true ground truth masks, despite having been trained exclusively on pseudo-labelled data from a different dataset. This cross-dataset generalisation confirms that the Attention U-Net has learned genuine crack features rather than overfitting to the specific noise patterns of the pseudo mask generation pipeline.

Metric	Value
Best Validation Dice	0.6915
Final Training Dice	0.7842
Final Training Loss	0.1722
Final Validation Loss	0.2702
Total Epochs	30
Input Resolution	448 × 448
Loss Function	Tversky Loss ($\alpha = 0.3$, $\beta = 0.7$)

Table 5.1: Summary of Training Metrics

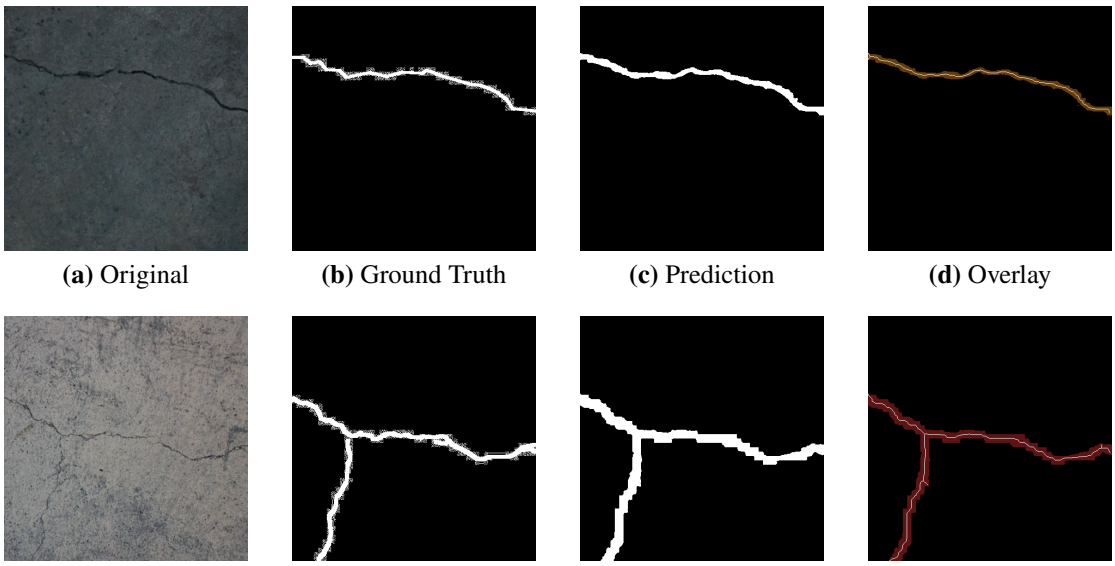
Images with higher crack area percentage and clearer crack contrast tended to achieve

higher IoU and Dice scores, as the model is more confident and spatially precise on prominent cracks. Images with very thin hairline cracks or heavily textured backgrounds tended to produce lower scores, as these are the cases where pseudo mask noise during training was highest and where the model received the least consistent supervision.

5.2.1 Sample Segmentation Outputs

The visual analysis of the generated masks from the test images highlights several qualities of the segmentation process.

When dealing with images featuring clearly visible cracks, usually more substantial and with a good contrast to the gray concrete background, the network provides neat masks following the contours of the cracks. The branching pattern of cracks is also identified when a network detects cracks with more than one arm meeting at a junction point; all cracks are kept without any omissions. This confirms that the skip connections and attention gates in the Attention U-Net successfully preserve fine spatial detail through the decoder.



(a) Original **(b)** Ground Truth **(c)** Prediction **(d)** Overlay

(e) Original **(f)** Ground Truth **(g)** Prediction **(h)** Overlay

Figure 5.3: Qualitative results of crack segmentation

In cases where the cracks are visible to some extent but not very clearly, there will be slight inaccuracies in the boundaries when predicting the masks for the cracks. These masks will accurately represent the location of the cracks, but the width will be somewhat incorrect, producing IoUs in the range of 0.4 to 0.6.

On images from pavement and bridge deck categories with complex surface textures, the model occasionally produces false positive predictions in regions with strong texture

edges or surface stains. The attention gates partially suppress these responses, but not entirely, particularly in cases where the texture pattern closely mimics the elongated dark structure of a crack. These cases are identified and analysed further using Grad-CAM in Section 5.4.

5.2.2 Failure Case Analysis

Failure cases defined as test images with predicted mask IoU below 0.3 against the ground truth fall into two broad categories.

The first category is false positive dominated failure, where the model predicts a significant crack mask on an image that contains no crack or only a very minor crack. In these cases the predicted crack area percentage is non-trivial but the ground truth mask is mostly empty, resulting in near-zero IoU. Grad-CAM analysis of these cases, presented in Section 5.4, reveals that the model's encoder is attending to surface stains, joint lines, and aggregate boundaries that share visual similarity with crack structures.

The second category is false negative dominated failure, where the model fails to predict the crack mask on an image that contains a clearly visible crack in the ground truth. These cases are less common and tend to occur on images where the crack is very thin, very low contrast, or located in a region of high surface texture complexity. In these cases the model's encoder activations are diffuse and low-confidence, indicating that the crack features were insufficient to trigger a strong segmentation response.

5.3 Severity Analysis Results

The severity analysis pipeline was run on all images in the test set. Each image was classified into one of five severity levels — NONE, LOW, MEDIUM, HIGH, or CRITICAL — based on the crack area percentage of its predicted mask. The severity distribution across the test set reflects both the true distribution of crack severity in the dataset and the model's prediction behaviour.

Images classified as NONE correspond to test images where the model predicted no crack pixels above the 0.5 threshold, which includes both correctly identified non-cracked images and cracked images where the model failed to detect the crack. The images that are labeled as LOW are those with the highest frequency of occurrence in line with the high number of hairline and surface cracks found within the set. Images that fall under the HIGH and CRITICAL categories are relatively few in number, indicating

the more serious occurrences.

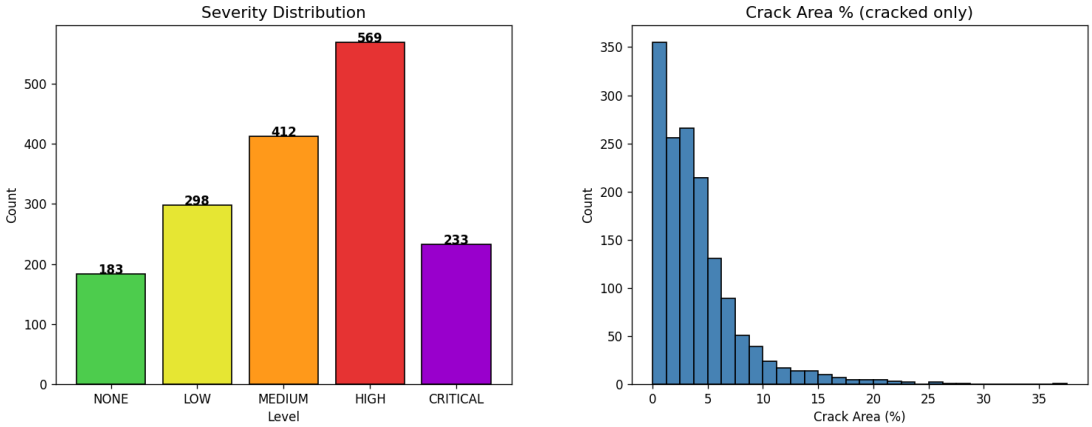


Figure 5.4: Severity analysis results across the full SDNET2018 test set

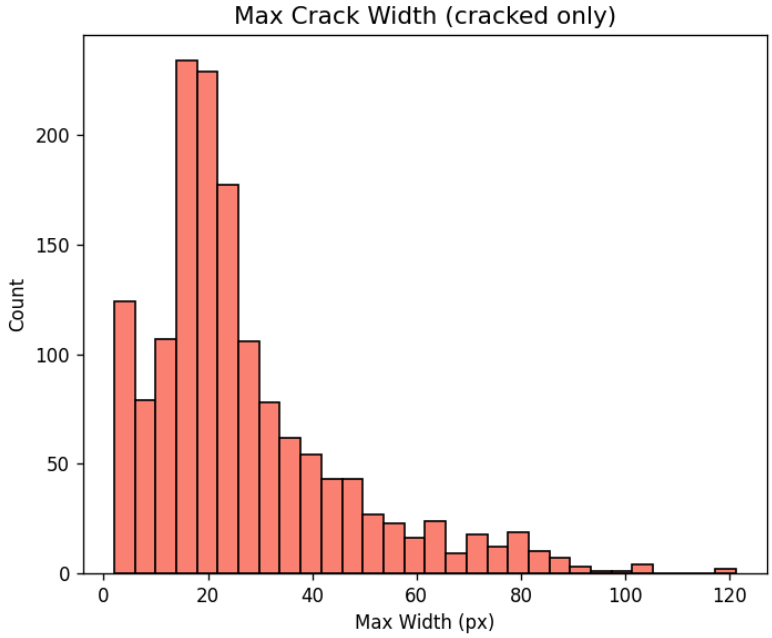


Figure 5.5: Distribution of maximum crack width across cracked test images

The severity distribution is recorded in the same CSV file where the other geometric measurements for each image are recorded. The severity distribution can then be used to further investigate the correlation between severity level and segmentation accuracy. A graph depicting the severity distribution for the test data set gives an overview of the findings from the inspections.

5.3.1 Per-Image Crack Geometry Metrics

In addition to the category of severity level, five different geometric measurements are calculated for each image in the pipeline. These are listed below with sample results

Table 5.2: Severity level distribution and mean crack metrics across the SDNET2018 test set (N=1,695 images).

Level	Area Threshold	Count	Pct. (%)	Mean Area (%)	Mean Max Width (px)	Mean Length (px)
NONE	= 0%	183	10.8	0.00	0.0	0
LOW	0.01–1%	298	17.6	0.35	8.1	179
MEDIUM	1–3%	412	24.3	1.98	20.4	458
HIGH	3–7%	569	33.6	4.56	28.3	637
CRITICAL	> 7%	233	13.7	11.45	57.1	897
Total / Mean	–	**1695**	100.0	3.67	22.0	434

seen in the testing data.

Percentage Crack Area varied between 0% for images that had no cracks and higher than 10% for images with severe cracks. Most images with cracks were between 0.1% and 3%, which correlates well with the crack widths included in the dataset.

Maximum Crack Width is expressed in pixels and reflects the widest point of the crack as measured by the distance transform at skeleton pixels. For hairline cracks, maximum widths of 2 to 5 pixels were typical at the 448×448 resolution. For structural cracks, maximum widths of 10 to 30 pixels were observed. These pixel values can be converted to physical millimetres if the physical scale of the image is known through camera calibration.

Mean Crack Width provides a more robust measure of overall crack width than the maximum, as it is less sensitive to localised wide regions at crack junctions. Mean widths were consistently lower than maximum widths, with the ratio of mean to maximum width providing an indication of crack uniformity means a crack with a high mean-to-maximum ratio is relatively uniform in width, while a low ratio indicates a crack that is narrow along most of its length with one or more wide junction points.

Crack Length in Pixels indicates the total length of the crack skeleton. If the cracks have long length but small width, then it can be assumed that the cracks are hairline cracks. However, if the length is shorter and the width is wide, the crack is considered to be structural damage.

Number of Crack Regions was typically 1 or 2 for images with a single dominant crack, and higher for images with multiple isolated cracks or diffuse surface crazing. A high region count combined with low individual crack widths is characteristic of surface crazing, which has different structural implications from a single deep structural crack of similar total area.

5.3.2 Sample Severity Overlay Visualisations

For each image used in the testing phase, there is a three-pane severity overlay visualization that will be created based on the batch processing pipeline, and will be stored together with the resulting CSV file in PNG format. The three-pane visualization consists of the original image, the binary mask image generated from the original image, and the overlaid image that shows cracks colored according to the severity level—NONE/LOW in green color, MEDIUM in orange color, HIGH in red color, and CRITICAL in purple color.

Visualisation plays a pivotal role in providing engineers with the main output from this technology. Instead of having to decipher a bare segmentation map on their own, an overlay can be shown that highlights the crack position, length, and severity in one image annotation.

5.4 Grad-CAM Analysis Results

5.4.1 Heatmap Visualisation on Successful Predictions

In the case when there are good test images for which the model gives a precise mask of cracks (with IoU > 0.5), the Grad-CAM heatmap shows clear activation localized around the crack region. The regions of the heatmap in warm colors perfectly coincide with the visible crack on the test image, as well as with the predicted crack region by the model itself. Thus, one can be sure that the mechanism of attention in the encoder has been realized correctly.

In the areas of the crack junctions and crack segments with large width, there are places where the activation becomes maximum. This agrees with the idea that the model has learned that those areas are crucial features for distinguishing cracks from other objects. On the other hand, thinner segments of cracks receive somewhat lower but distinct activation compared to the background.

This demonstrates that the attention gate works properly, and for the successful prediction, encoder activations responsible for segmenting are precisely localized in the crack region on the image.

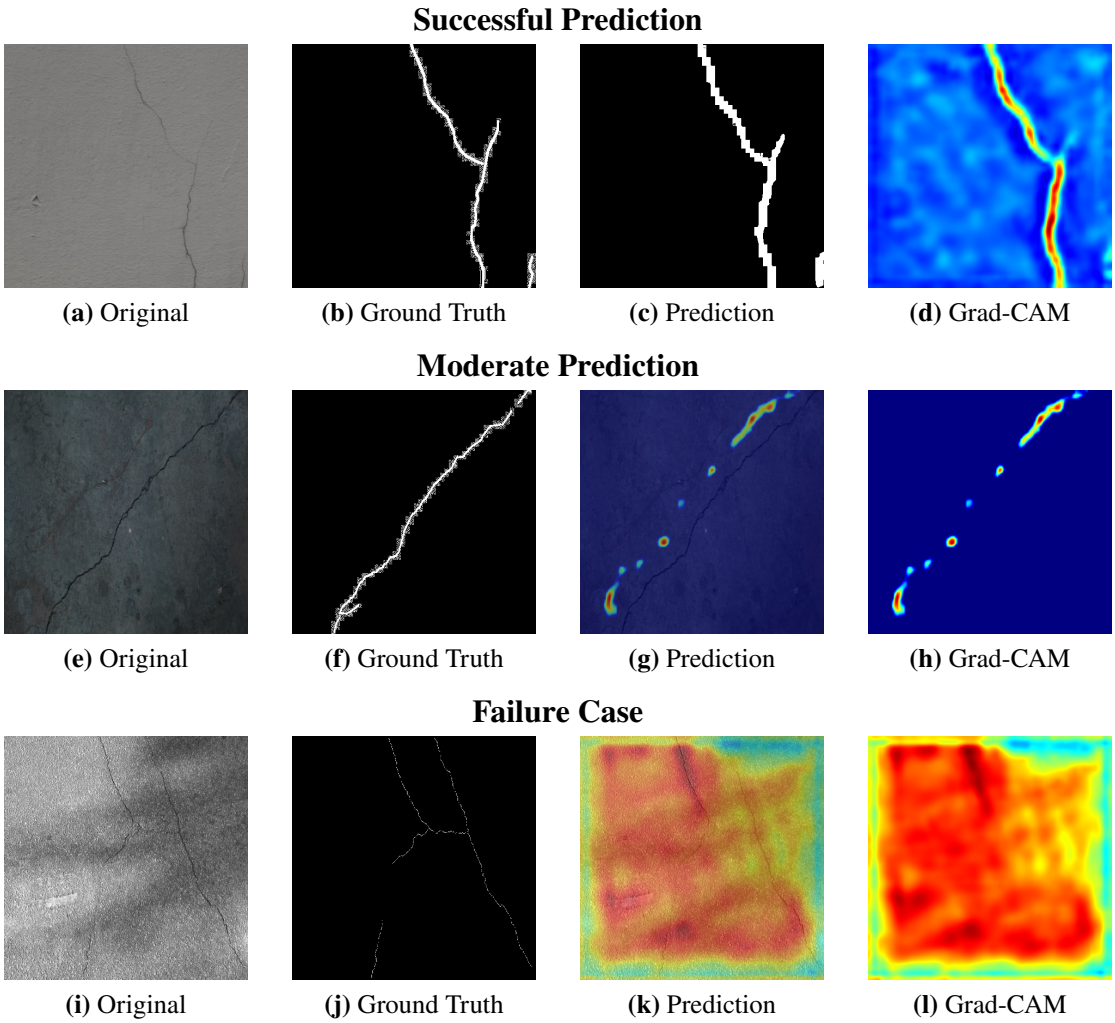


Figure 5.6: Grad-CAM visualisation of the Attention U-Net model across different prediction scenarios. The top row shows a successful prediction with strong attention focused along the crack structure. The middle row shows a moderate prediction where the heatmap highlights both crack regions and some background noise, indicating uncertainty. The bottom row shows a failure case where attention is dispersed over irrelevant textures, leading to misclassification.

5.4.2 Heatmap Visualisation on Failure Cases

On failure cases with IoU below 0.3, the Grad-CAM heatmaps reveal distinctly different activation patterns that explain the nature of the prediction error.

For false positive dominated failure cases, the model predicts cracks on a non-cracked or lightly cracked image and the heatmap shows strong activation concentrated on surface stains, dark aggregate inclusions, joint lines between pavement sections, or elongated shadow regions. These are features that share the visual characteristics of cracks — dark, elongated, relatively narrow structures with a contrast gradient at their boundaries — but are not structural cracks. The model's encoder responds to these features because the pseudo mask generation pipeline during training also produced false positive mask regions on similar features in the training set, causing the model to learn that such visual patterns are associated with crack labels.

False negative dominated failure cases occur where the model misses a visible crack and the heatmap shows diffuse, spatially spread activation without a clear peak at the crack location. It means that the activation of the encoder neurons around the crack area is not sufficiently strong or differs insignificantly from that of the surrounding neurons, making it difficult for the algorithm to generate a definite segmentation response. Such scenarios usually happen when there is a crack that contrasts very little against its surroundings, like a crack on a rough surface of a bridge deck.

5.4.3 Observations and Implications

The Grad-CAM analysis yields three principal observations that have direct implications for future model development.

First, surface stains are the primary driver of false positive predictions. The heatmap evidence shows consistently that false positives are caused by the model attending to stain boundaries rather than crack structures. This suggests that a pre-processing step to suppress stain regions—for example, using colour-based segmentation to identify discoloration patterns—could significantly reduce the false positive rate.

Second, pseudo mask noise during training is the root cause of the false positive learning behaviour. Since the pseudo mask generation pipeline produced false positive mask regions on stain boundaries in the training data, the model learned to associate stain boundaries with crack labels [9]. This would not occur if true pixel-level annotations were used for training. Manual annotation of even a modest number of training images

— particularly for challenging surface types like pavements—would likely improve false positive performance substantially.

Third, the Grad-CAM analysis confirms that the Attention U-Net's attention gates are working correctly on successful predictions but are insufficient on their own to eliminate false positive responses to stain boundaries on failure cases. This suggests that incorporating explicit stain-suppression or texture-normalisation into the preprocessing pipeline would be a more effective intervention than architectural changes to the model.

Training dynamics, segmentation scores on the Kaggle dataset, severity statistics on the test set, individual image crack statistics, and Grad-CAM results for both success and failure cases have been covered in this chapter. The network attains its best validation Dice score at 0.6915 and is able to generalise to new unseen images with actual masks even though it was only trained on pseudo-labeled data. From the Grad-CAM results, the network fails due to surface stains and pseudo-labeled data noise. Chapter 6 summarises the findings and future directions.

CHAPTER 6

CONCLUSION

The research is aimed at the development of a deep learning technique capable of detecting cracks on different textured surfaces used in infrastructure construction. The key idea behind the suggested method involves integrating pseudo mask generation algorithm into the system workflow. This system uses a pseudo mask creation pipeline within its process. The proposed system integrates a pseudo mask generation pipeline [7, 9], an Attention U-Net segmentation model [16] combined with the use of Tversky loss function [21] and mixed precision training [22], severity analysis based on geometry and Grad-CAM visualization [18]. As an input image data, authors used a specially selected multi-textured images dataset provided by SDNET2018 [1] and successfully detected cracks in complicated visual conditions.

In summary, the results indicate that an explainable deep learning model can considerably boost the effectiveness of crack detection applications in structural inspection. The best validation Dice of 0.6915, the five-level severity classification, and the Grad-CAM-based failure diagnostics together demonstrate that the proposed system provides structural engineers with not only crack localisation but also actionable severity information and model transparency. With the improvements incorporated, the system demonstrates considerable promise for facilitating quick, accurate, and reliable structural assessments [23].

Despite its success in current operations, there is still an opportunity to optimize this approach. Some future considerations include: (1) replacing pseudo masks with a hand-labeled set of samples to reduce the number of false positives in the regions where the stains occur; (2) testing new network types such as UNet++ [24], as well as transformer encoder models [25]; (3) introducing calibration for cameras to convert pixel values of cracks' widths into actual millimeters; and (4) expanding to support video feeds or real-time inspections on mobile platforms.

BIBLIOGRAPHY

[1] S. Dorafshan, R. J. Thomas, and M. Maguire, "SDNET2018: An annotated image dataset for non-contact concrete crack detection using deep convolutional neural networks," *Data in Brief*, vol. 21, pp. 1664–1668, 2018.

[2] C. Fan *et al.*, "A review of crack research in concrete structures based on data-driven and intelligent algorithms," *Structures*, 2025.

[3] Y. Nomura, M. Inoue, and H. Furuta, "Evaluation of crack propagation in concrete bridges from vehicle-mounted camera images using deep learning and image processing," *Frontiers in Built Environment*, vol. 8, September 2022.

[4] Y. Li, "Crack detection and semantic segmentation in concrete structures using an improved deep learning-based framework," *Journal of Engineering and Applied Science*, vol. 73, p. 107, March 2026.

[5] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *Lecture Notes in Computer Science (MICCAI)*, vol. 9351, pp. 234–241, 2015.

[6] A. Dickson and C. Thomas, "Improved PSO for optimizing the performance of intrusion detection systems," *Journal of Intelligent & Fuzzy Systems*, vol. 38, no. 1, pp. 547–560, 2020.

[7] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.

[8] F. Zhao, Y. Chao, and L. Li, "A crack segmentation model combining morphological network and multiple loss mechanism," *Sensors*, vol. 23, no. 3, p. 1127, January 2023.

[9] A. F. Frangi, W. J. Niessen, K. L. Vincken, and M. A. Viergever, "Multiscale vessel enhancement filtering," *Lecture Notes in Computer Science (MICCAI)*, vol. 1496, pp. 130–137, 1998.

[10] Y.-J. Cha, W. Choi, and O. Büyüköztürk, "Deep learning-based crack damage detection using convolutional neural networks," *Computer-Aided Civil and Infrastructure Engineering*, vol. 32, no. 5, pp. 361–378, 2017.

[11] S. Li and X. Zhao, "Image-based concrete crack detection using convolutional neural network and exhaustive search technique," *Advances in Civil Engineering*, vol. 2019, pp. 1–12, 2019.

[12] N. Sharma, R. Dhir, and R. Rani, "Crack detection in concrete using transfer learning," *Advances in Mathematics: Scientific Journal*, vol. 9, no. 6, pp. 3895–3906, 2020.

[13] F. Chen and M. R. Jahanshahi, "Nb-cnn: Deep learning-based crack detection using convolutional neural network and naïve bayes data fusion," *IEEE Transactions on Industrial Electronics*, vol. 65, no. 5, pp. 4392–4400, 2018.

[14] V. P. Golding, Z. Gharineiat, H. S. Munawar, and F. Ullah, "Crack detection in concrete structures using deep learning," *Sustainability*, vol. 14, no. 13, p. 8117, 2022.

[15] S. S. Zadeh, S. A. Birgani, M. Khorshidi, and F. Kooban, "Concrete surface crack detection with convolutional-based deep learning models," *International Journal of Novel Research in Civil Structural and Earth Sciences*, vol. 10, no. 3, pp. 25–35, 2023.

[16] O. Oktay, J. Schlemper, L. L. Folgoc, M. Lee, M. Heinrich, K. Misawa, K. Mori, S. McDonagh, N. Y. Hammerla, B. Kainz, B. Glocker, and D. Rueckert, "Attention u-net: Learning where to look for the pancreas," *arXiv preprint arXiv:1804.03999*, 2018.

[17] K. N. Vankudothu and W. Bukaita, "Quantitative analysis of crack growth and severity in reinforced concrete structures using deep learning and computer vision," *American Journal of Traffic and Transportation Engineering*, vol. 10, no. 6, pp. 150–167, December 2025.

[18] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

[19] Wikipedia contributors, "Bilateral filter," https://en.wikipedia.org/wiki/Bilateral_filter, 2024, accessed: 2024.

[20] Analytics Vidhya, "Image contrast enhancement using CLAHE," https://www.analyticsvidhya.com/blog/2022/08/image-contrast-enhancement-using-clahe/, August 2022.

[21] S. S. M. Salehi, D. Erdogmus, and A. Gholipour, "Tversky loss function for image segmentation using 3D fully convolutional deep networks," *Lecture Notes in Computer Science (MLMI at MICCAI)*, vol. 10541, pp. 379–387, 2017.

[22] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh *et al.*, "Mixed precision training," *arXiv preprint arXiv:1710.03740*, 2018. [Online]. Available: https://arxiv.org/abs/1710.03740

[23] T. S. Tran, S. D. Nguyen, H. J. Lee, and V. P. Tran, "Advanced crack detection and segmentation on bridge decks using deep learning," *Construction and Building Materials*, vol. 400, p. 132839, 2023.

[24] Z. Zhou, M. M. Rahman Siddiquee, N. Tajbakhsh, and J. Liang, "UNet++: A nested U-Net architecture for medical image segmentation," *Lecture Notes in Computer Science (DLMIA at MICCAI)*, vol. 11045, pp. 3–11, 2018.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

136 Not Cited or Quoted 12%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
10% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.